

MPSI – Tematy projektów semestralnych

Informacje organizacyjne

Każdy zespół powinien:

- samodzielnie zaimplementować wymagane algorytmy,
- przeprowadzić eksperymenty,
- przygotować analizę wyników,
- stworzyć raport w formie *Jupyter Notebook*,
- omówić ograniczenia oraz problemy napotkane podczas projektu.

Sugestie techniczne

Do implementacji można wykorzystać:

- Python,
- NumPy,
- matplotlib,
- scikit-learn,
- pandas,
- Streamlit,
- Jupyter Notebook.

Projekty 1-osobowe

1. Detekcja anomalii w transakcjach finansowych

Cel projektu:

Wykrywanie podejrzanych transakcji przy użyciu algorytmu *k-means*.

W projekcie należy:

- przygotować oraz znormalizować dane,
- zaimplementować algorytm *k-means*,
- wykorzystać odległość od centroidu do wykrywania anomalii,
- przeanalizować wpływ progu odcięcia,
- obliczyć:
 - accuracy,
 - precision,
 - recall,
 - F1-score,
- przygotować macierz błędów,
- narysować krzywą ROC.

Przykładowe dane:

- Kaggle Credit Card Fraud Detection.

2. Implementacja kodera rANS

Cel projektu:

Implementacja uproszczonego kodera i dekodera rANS oraz analiza zależności pomiędzy entropią a długością kodu.

W projekcie należy:

- zaimplementować kodowanie i dekodowanie rANS,
- sprawdzić poprawność rekonstrukcji danych,
- przeanalizować wpływ rozkładu symboli na długość kodu,
- obliczyć entropię źródła,
- porównać rzeczywistą długość kodu z wartością teoretyczną,
- przeanalizować działanie renormalizacji.

Przykładowe dane:

- Canterbury Corpus,
- Calgary Corpus,
- krótkie pliki tekstowe.

3. Analiza entropii krzyżowej i dywergencji KL

Cel projektu:

Badanie wpływu błędnego modelu probabilistycznego na długość kodu.

W projekcie należy:

- wygenerować dane z rozkładu $p(x)$,
- zdefiniować alternatywny rozkład $q(x)$,
- obliczyć:
 - entropię $H(p)$,
 - entropię krzyżową,
 - dywergencję KL,
- porównać wyniki eksperymentalne z teorią,
- przeanalizować wpływ błędnego modelu na długość kodu.

4. Wizualizacja działania algorytmu genetycznego

Cel projektu:

Wizualizacja procesu ewolucji populacji podczas działania algorytmu genetycznego.

W projekcie należy:

- zaimplementować prosty algorytm genetyczny,
- zaimplementować:
 - selekcję,
 - mutację,
 - crossover,
- wizualizować populację w kolejnych iteracjach,
- przeanalizować wpływ parametrów:
 - mutation rate,
 - population size,
 - crossover probability,
- porównać szybkość zbieżności dla różnych ustawień.

Przykładowe funkcje celu:

- Rastrigin,
- Rosenbrock.

Projekty 2-osobowe

1. Segmentacja obrazów dla pojazdów autonomicznych

Cel projektu:

Segmentacja obrazów przy użyciu algorytmu *k-means*.

W projekcie należy:

- rozszerzyć opis piksela o współrzędne (x, y) ,
- porównać segmentację:
 - tylko na podstawie koloru,
 - z uwzględnieniem położenia pikseli,
- przeanalizować wpływ liczby klastrów,
- ocenić jakość segmentacji przy użyciu IoU,
- przygotować wizualizacje segmentacji.

Przykładowe dane:

- Cityscapes Dataset.

2. Segmentacja klientów sklepu internetowego

Cel projektu:

Grupowanie klientów na podstawie zachowań zakupowych.

W projekcie należy:

- przygotować oraz oczyścić dane,
- dobrać liczbę klastrów metodą łokcia,
- przeanalizować cechy poszczególnych grup klientów,
- sprawdzić stabilność podziału,
- przygotować wizualizacje:
 - PCA,
 - t-SNE,
 - wykresy radarowe.

Przykładowe dane:

- UCI Online Retail Dataset.

3. Porównanie Huffmana, Arithmetic Coding i rANS

Cel projektu:

Porównanie skuteczności różnych metod kodowania entropijnego.

W projekcie należy:

- zaimplementować co najmniej dwie metody kodowania,
- porównać:
 - długość kodu,
 - czas kodowania,
 - czas dekodowania,
- przeanalizować wpływ rozkładu danych,
- porównać wyniki z teoretyczną entropią źródła,
- przygotować wykresy porównawcze.

Przykładowe dane:

- Canterbury Corpus,
- Project Gutenberg,
- MNIST.

4. Algorytm genetyczny dla problemu komiwojażera

Cel projektu:

Rozwiązanie problemu TSP przy użyciu algorytmów genetycznych.

W projekcie należy:

- zakodować trasę jako chromosom,
- zaimplementować:
 - mutacje,
 - crossover,
 - selekcję,
- porównać różne operatory crossover,
- analizować długość znalezionych tras,

- wizualizować przebieg optymalizacji.

Przykładowe dane:

- TSPLIB.

5. Optymalizacja hiperparametrów modelu ML

Cel projektu:

Automatyczny dobór hiperparametrów modelu uczenia maszynowego.

W projekcie należy:

- wybrać model ML (preferowany jakiś prosty model płytki),
- zakodować hiperparametry jako chromosom,
- wykorzystać algorytm genetyczny do optymalizacji,
- porównać wyniki z:
 - grid search,
 - random search,
- przeanalizować koszt obliczeniowy metod.

Przykładowe dane:

- MNIST,
- CIFAR-10,
- Titanic.

6. Modelowanie tematów w NLP

Cel projektu:

Grupowanie dokumentów tekstowych według tematyki.

W projekcie należy:

- przygotować reprezentację TF-IDF,
- zastosować algorytm *k-means*,
- znaleźć słowa najlepiej opisujące klastry,
- przeanalizować wpływ liczby klastrów,
- przygotować wizualizację metodą t-SNE.

Przykładowe dane:

- 20 Newsgroups.

7. Analiza danych genomowych

Cel projektu:

Klasteryzacja danych biologicznych oraz analiza wpływu wysokiego wymiaru i wartości odstających.

W projekcie należy:

- przygotować dane genomowe,
- porównać różne metryki odległości,
- przeanalizować wpływ outlierów,
- przygotować heatmapy,
- przeanalizować stabilność klastrow.

Przykładowe dane:

- TCGA,
- GEO.

8. Porównanie algorytmów gradientowych i ewolucyjnych

Cel projektu:

Porównanie metod gradientowych z algorytmami ewolucyjnymi.

W projekcie należy:

- zaimplementować:
 - gradient descent,
 - algorytm genetyczny,
- porównać szybkość zbieżności,
- przeanalizować odporność na minima lokalne,
- porównać działanie dla różnych funkcji celu,
- przygotować wykresy przebiegu optymalizacji.

Przykładowe funkcje celu:

- Rastrigin,
- Ackley,
- Rosenbrock.

Ocena projektu

Pod uwagę będą brane:

- poprawność implementacji,
- jakość eksperymentów,
- analiza wyników,
- jakość raportu,
- prezentacja projektu.